

Moteur de formulaire Runique

Trois patterns GoF, trois niveaux

Strategy · Composite · Template Method

La composition

Template Method	RuniqueForm::is_valid/save		flux figé
Composite	Forms (agrège les champs)		le tout délègue
Strategy	FormField (le champ)		algo interchangeable

Trois patterns, trois niveaux d'abstraction.

1 · Strategy — le champ `FormField`

Intention : une famille d'algorithmes interchangeable derrière une interface commune, choisis à l'exécution.

```

    trait FormField (Strategy)
    + validate() -> bool
    + render(tera) -> Result<String>
    + finalize() (hook, défaut no-op)

    TextField  ─┐
               ▲
    ChoiceField ┤
               ▲
    FileField  ┤
               ▲
    HiddenField ┤
               ▲
    (ConcreteStrategy)
  
```

Context : `Forms.fields : Map<String, Box<dyn FormField>>`

1 · Strategy — ce que fait `validate()`

Champ	Règles
<code>TextField</code>	required, longueur min/max, format (Email/Url/Password)
<code>NumericField</code>	parse Int/Float/Decimal, bornes, step
<code>ChoiceField</code>	valeur ∈ <code>Vec<ChoiceOption></code>
<code>FileField</code>	type, extensions, taille vs <code>model_max_size</code>
<code>HiddenField</code>	CSRF : <code>ct_eq</code> si <code>expected_value</code>

`render()` : chaque champ a un `template_name` , construit son contexte Tera depuis `FieldConfig` , rend son fragment HTML.

1 · Strategy — idiome Rust

- `Box<dyn FormField>` = **trait object** (dispatch dynamique), pas d'héritage.
- Super-trait `DynClone` : Rust ne clone pas un `Box<dyn Trait>` nativement → c'est ce qui permet `#[derive(Clone)]` sur `Forms`.
- `finalize()` : défaut `ok(())`, surchargé par ex. le champ password (hash après validation).

2 · Composite — le formulaire **Forms**

Intention : traiter uniformément l'ensemble et l'unité, le conteneur déléguant à ses parties puis agrégeant.

Validation :

```
Forms::is_valid()
├─ court-circuit si force_invalid (honeypot)
├─ validated = true
└─ Forms::validate() — garde de profondeur (StackOverflow)
    └─ FormValidator::validate_fields()
        for field in fields { field.validate() } → tous valides ?
```

2 · Composite — rendu & écart

Rendu :

```
Forms::render() → FormRenderer::render(fields, errors)
    for field in fields.values() { field.render(tera) }
    └─ concatène les fragments HTML
```

Écart au manuel : Composite **plat** (Form → champs, pas de récursivité).

L'intention (agrégation + délégation) est là ; la récursivité n'est pas nécessaire.

3 · Template Method — `is_valid()`

Intention : squelette figé + étapes remplies par des **hooks**.

```
RuniqueForm::is_valid()    (ordre VERROUILLÉ)
  1. is_submitted() ?      → false sans erreur si non
  2. Forms::is_valid()      → valide chaque champ (composite)
  3. clean_field() par champ (hook)
  4. clean()               (hook métier)
      ok → finalize() → true
      err → répartit les erreurs sur les champs → false
```


3 · Template Method — `save()` & idiome

```
save()  (squelette transactionnel)
  ouvre txn
  |
  |— before_save()    (hook)
  |— on_save()        (hook — écriture métier)
  |— after_save()     (hook)
  |— Err → rollback (tracé) | Ok → commit
```

Idiome Rust : les **méthodes de trait par défaut** *sont* les primitive operations. Le trait fournit le squelette ; le form concret ne surcharge que les hooks. (Équivalent du `clean()` de Django.)

Synthèse

Pattern	Portée	Ce qui varie
Strategy	un champ	l'algo <code>validate</code> / <code>render</code> par type
Composite	le formulaire	rien — délègue + agrège
Template Method	le flux	les hooks métier, pas l'ordre

Tradeoff : validation au **runtime** (pas à la compilation).

→ Ajouter un type de champ = une stratégie de plus, sans impact sur conteneur ni flux ;
flux métier verrouillé par le trait.